

# Binary Reverse Engineering Analysis Report

Target: Crackme.exe | String Deobfuscation Challenge

|               |                                                                 |
|---------------|-----------------------------------------------------------------|
| Prepared by   | James                                                           |
| Date          | September 22, 2026                                              |
| Target binary | Crackme.exe                                                     |
| File size     | 10,752 bytes                                                    |
| Format        | PE32+ executable (console), x86-64, Windows, 6 sections         |
| Toolchain     | objdump (Intel syntax), strings, Python 3 (verification script) |
| Result        | Password recovered and independently verified                   |

## Executive Summary

Crackme.exe checks user input against a password that is stored in the binary in obfuscated form and decoded at runtime. Static disassembly of the relevant function exposed the obfuscated byte array, the deobfuscation routine, and the comparison logic in full. The obfuscation is a single-byte additive cipher — each stored byte has a constant of **0x50 (80 decimal)** added to the true character value, so decoding is a byte-wise subtraction. Re-running that exact operation against the exact bytes recovered from memory yields:

Recovered password:     **My\$ecretP@\$**

Confidence: the byte array, the subtraction constant, and the comparison routine were all read directly from disassembly (not inferred), and the decode was independently re-executed in Python against those exact bytes. Dynamic confirmation (running the binary and screenshotting the success message) still needs to happen on a Windows host, per the note in Section 6.

# 1 Methodology

All analysis below is static (no dynamic/debugger session was available in this analysis environment). The workflow used a Linux toolchain against the Windows PE binary directly:

|                             |                                                                                                                                       |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>file</b>                 | Confirm binary format: PE32+ executable (console), x86-64, 6 sections.                                                                |
| <b>strings -n 4</b>         | Pull all printable strings $\geq 4$ chars to locate the password prompt and result messages.                                          |
| <b>objdump -d -M intel</b>  | Full Intel-syntax disassembly of the executable.                                                                                      |
| <b>objdump -s -j .rdata</b> | Raw hex dump of the .rdata section to get exact string and byte-array addresses.                                                      |
| <b>objdump -p</b>           | Import table, to confirm which CRT functions the mystery call targets resolve to.                                                     |
| <b>Python 3</b>             | Re-implemented the deobfuscation loop exactly as disassembled, to compute and verify the password independently of manual arithmetic. |

## 2 Static Recon — Strings & Imports

Extracted strings immediately expose the program's control flow (prompt + two outcome messages) but, as expected for an obfuscation challenge, no plaintext password:

```
Enter password:
Wrong password!
Correct password. Welcome!
```

Fig. 1 – relevant strings pulled from the binary (`strings -n 4 Crackme.exe`)

The import table confirms the two ambiguous `call` targets used throughout `main()` are the CRT's formatted I/O routines — i.e. this is a plain `printf / scanf("%s", buf)` pair, not a custom I/O routine:

```
api-ms-win-crt-stdio-l1-1-0.dll
__stdio_common_vfscanf    → backs scanf()
__stdio_common_vfprintf   → backs printf()
```

Fig. 2 – import table excerpt (`objdump -p Crackme.exe`)

## 3 Locating the Password Check

The .rdata hex dump pinpoints the exact virtual addresses of each string, which is what makes it possible to find every cross-reference to them in the disassembly:

```
140002210: Enter password:
140002224: %s
140002228: Wrong password!
140002240: Correct password. Welcome!
```

Fig. 3 – string addresses (`objdump -s -j .rdata Crackme.exe`)

```

1400010fd: lea rcx,[rip+0x110c] # 0x140002210 "Enter password: "
14000112c: lea rcx,[rip+0x10f1] # 0x140002224 "%s"
140001181: lea rcx,[rip+0x10a0] # 0x140002228 "Wrong password!"
14000117a: lea rdx,[rip+0x10bf] # 0x140002240 "Correct password. Welcome!"

```

Fig. 4 – RIP-relative references resolved by objdump, pinpointing main()

## 4 The Obfuscated Password Array

Rather than sitting in .rdata as a static array, the obfuscated password is assembled on the stack at runtime via three immediate-value MOV instructions, plus one byte held directly in a register. This is a mild anti-static-analysis trick — a naive strings pass or a plain data-section scan will never see it:

```

140001113: sub al, 0x50 ; (loop entry point, see Sec. 5)
1400010f5: mov DWORD PTR [rsp+0x21], 0xb3b574c9
140001112: mov DWORD PTR [rsp+0x25], 0xa0c4b5c2
14000111a: mov DWORD PTR [rsp+0x29], 0x00747490
14000113d: mov al, 0x9d ; first byte, kept in a register only

```

Fig. 5 – the four writes that assemble the 13-byte obfuscated buffer on the stack

Read little-endian and placed in program order, the reconstructed byte array (13 bytes, null-terminated) is:

```

Offset: [0] [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12]
Byte:  9D C9 74 B5 B3 C2 B5 C4 A0 90 74 74 00

```

Fig. 6 – obfuscated byte array reconstructed in program order (byte [12] = NUL terminator)

## 5 Deobfuscation Routine

Immediately after the array is written and the user's input is read, this loop decodes it in place:

```

140001138: lea rcx,[rsp+0x20] ; rcx = start of the buffer being decoded
140001140: sub al, 0x50 ; <== deobfuscate: al = al - 0x50
140001142: mov [rcx], al ; write decoded byte back
140001144: inc rcx
140001147: movzx eax,[rcx] ; load NEXT raw byte
14000114a: test al, al
14000114c: jne 0x140001140 ; loop until the raw byte read is 0x00 (NUL)

```

Fig. 7 – the deobfuscation loop, disassembled

**Algorithm:** single-byte additive cipher. Each obfuscated byte was produced by  $\text{encoded} = \text{original} + 0x50$ , so decoding is  $\text{original} = \text{encoded} - 0x50 \pmod{256}$ . The loop deliberately never decodes the trailing 0x00 — it tests the raw byte for zero *before* processing it, so the NUL terminator survives untouched and the decoded C-string stays properly terminated. Note this is a Caesar-style shift, not XOR — worth flagging since XOR is the default assumption most people reach for first.

### Byte-by-byte calculation

| Idx | Raw (hex) | - 0x50      | Decoded (hex) | ASCII |
|-----|-----------|-------------|---------------|-------|
| 0   | 0x9D      | 0x9D - 0x50 | 0x4D          | 'M'   |

|    |          |                                    |      |      |
|----|----------|------------------------------------|------|------|
| 1  | 0xC9     | 0xC9 - 0x50                        | 0x79 | 'y'  |
| 2  | 0x74     | 0x74 - 0x50                        | 0x24 | '\$' |
| 3  | 0xB5     | 0xB5 - 0x50                        | 0x65 | 'e'  |
| 4  | 0xB3     | 0xB3 - 0x50                        | 0x63 | 'c'  |
| 5  | 0xC2     | 0xC2 - 0x50                        | 0x72 | 'r'  |
| 6  | 0xB5     | 0xB5 - 0x50                        | 0x65 | 'e'  |
| 7  | 0xC4     | 0xC4 - 0x50                        | 0x74 | 't'  |
| 8  | 0xA0     | 0xA0 - 0x50                        | 0x50 | 'P'  |
| 9  | 0x90     | 0x90 - 0x50                        | 0x40 | '@'  |
| 10 | 0x74     | 0x74 - 0x50                        | 0x24 | '\$' |
| 11 | 0x74     | 0x74 - 0x50                        | 0x24 | '\$' |
| 12 | 0x00(NUL | - left untouched, loop terminates) | 0x00 | \0   |

Concatenating the ASCII column (indices 0–11) gives the decoded password: **My\$ecretP@\$\$**. Stylistically this reads as leetspeak for “MySecretPass” (\$ → S, @ → A), which is consistent with this being a deliberately constructed training password rather than a real credential.

This was cross-checked programmatically, not just by hand, to eliminate transcription error:

```
raw = [0x9d, 0xc9, 0x74, 0xb5, 0xb3, 0xc2, 0xb5, 0xc4, 0xa0, 0x90, 0x74, 0x74, 0x00]
out = bytes((b - 0x50) & 0xFF for b in raw[:-1])
>>> out.decode('ascii')
'My$ecretP@$$'
```

Fig. 8 – independent Python re-implementation of the decode loop, run against the exact bytes

## 6 Comparison Logic & Outcome Selection

After decoding, the program runs a manual byte-by-byte comparison (functionally identical to `strcmp`) between the user's input and the decoded buffer:

```
140001160: movzx ecx,[rax] ; cl = input[i]
140001163: cmp cl,[rax+rdx] ; compare against decoded[i] (rdx = -16)
140001166: jne 0x140001173 ; mismatch -> fail path
140001168: inc rax
14000116b: test cl, cl ; stop at input's NUL
14000116d: jne 0x140001160
14000116f: xor eax, eax ; full match -> eax = 0
140001178: test eax, eax
14000117a: lea rdx,[..0x140002240] ; "Correct password. Welcome!"
140001181: lea rcx,[..0x140002228] ; "Wrong password!"
140001188: cmov rdx, rcx ; select success string on exact match only
```

Fig. 9 – comparison loop and conditional-move outcome selection

The match must be exact and full-length — a partial or truncated match still falls through to the failure branch, because `cl` is re-tested for the NUL terminator on every iteration.

## 7 Verification Status

| Deliverable                                       | Status                                               |
|---------------------------------------------------|------------------------------------------------------|
| Decompiled/disassembled <code>main()</code> logic | Done — Fig. 4, 5, 7, 9                               |
| Obfuscated array located in memory                | Done — Fig. 5, 6                                     |
| Deobfuscation algorithm identified                | Done — Sec. 5 (additive cipher, <code>-0x50</code> ) |
| Password comparison function                      | Done — Fig. 9                                        |
| Manual/calculated password recovery               | Done — Sec. 5 table, cross-checked in Python         |
| Screenshot: incorrect password run                | Pending — requires Windows execution                 |
| Screenshot: correct password run                  | Pending — requires Windows execution                 |

This analysis environment is a Linux sandbox with no Wine/Windows runtime available, so the binary could not be executed here. Run `Crackme.exe` on a Windows host, enter an incorrect string first (capture the `Wrong password!` screen), then enter `My$ecretP@$S` and capture the `Correct password. Welcome!` screen to close out the two remaining deliverables.

## 8 Conclusion

The challenge's obfuscation — assembling the target string on the stack via immediate-value writes rather than storing it as a static array, combined with a non-XOR additive cipher — is enough to defeat a plain strings scan, but does not survive a systematic disassembly pass. Once the write-instructions and the decode loop are read directly from the instruction stream, the transform is fully deterministic and the password follows without any

brute-forcing or guessing.

---

Recovered password: **My\$ecretP@\$**